Collaborer avec Git et GitHub

Réunir trois branches sans casser main

Séquence
SQ01

Partie
P02

Niveau
Mastère ESI

Objectifs

- Intégrer son travail par une pull request relue
- Relire le code d'un pair et argumenter une demande de modification
- Résoudre un conflit de fusion et expliquer la version retenue

Prérequis : `clone`, `commit`, `push` ; les trois branches du matin.

ATTENTION

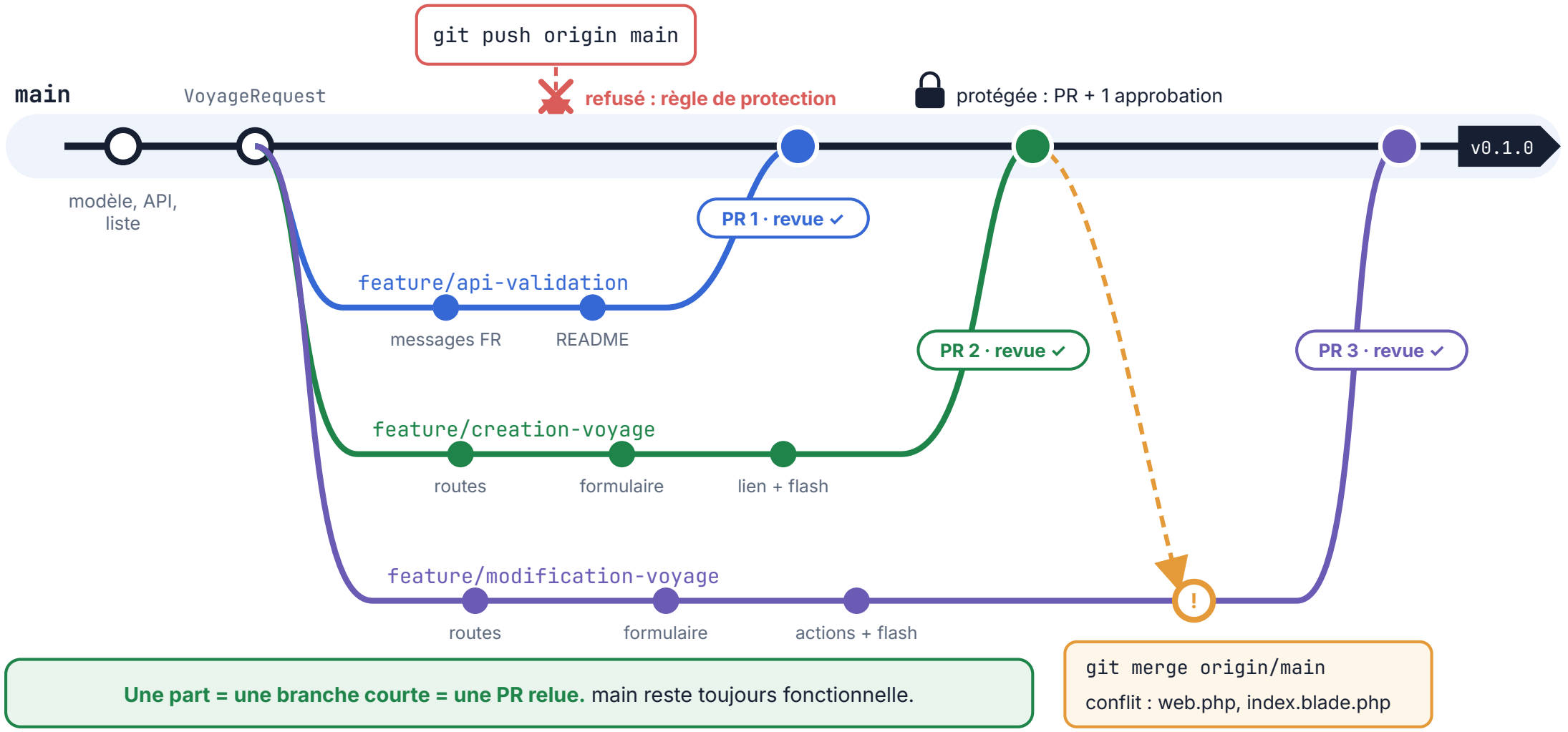
Le problème de l'après-midi

Trois branches, trois auteurs, un seul `main`.

Deux branches ont modifié `routes/web.php`, `VoyageController.php` et `index.blade.php`.

Comment tout réunir sans écraser personne, ni casser la version qui marche ?

Le GitHub flow, appliqué à ce matin



Ordre des fusions : 1. API · 2. Création · 3. Modification (dernière : elle intègre d'abord main)

GitHub flow ou Gitflow ?

	GitHub flow	Gitflow
Branches durables	`main` seule	`main`, `develop`
Branches de travail	une par changement	`feature/*`, `release/*`, `hotfix/*`
Livraison	continue, depuis `main`	par versions planifiées
Adapté à	petite équipe, CI/CD	cycles de version longs

main ne casse jamais

- Ce qui est dans `main` sera **testé, construit, déployé** automatiquement
- Donc : pas de push direct, **une PR et une approbation** pour chaque fusion
- La règle est technique, pas une promesse : GitHub refuse le reste

Démonstration : création du *ruleset* sur le dépôt, puis un push direct refusé.

Une PR se décrit

```
## Quoi
Ce que la branche ajoute, en deux ou trois phrases.

## Pourquoi
Le besoin ou le problème réglé.

## Comment tester
Les commandes et les gestes pour vérifier,
y compris une saisie invalide.
```

Le relecteur doit pouvoir tester **sans demander à l'auteur**.

Relire, ce n'est pas survoler

- **Tester** la branche sur son poste, pas seulement lire le diff
- **Comprendre** chaque ligne, ou poser la question
- **Traquer** les fichiers en trop : `.env`, `vendor/`, base SQLite
- **Commenter** en trois temps : constat, raison, proposition

> *« `store()` utilise `$request->all()` : un champ non prévu serait enregistré.*
> *Proposition : `validated()`. »*

Le conflit arrive au dernier

```
git switch feature/modification-voyage
git fetch origin
git merge origin/main
```

```
CONFLICT (content): Merge conflict in routes/web.php
Automatic merge failed; fix conflicts and then commit the
result.
```

On résout **sur son poste**, pour pouvoir tester avant de publier.

Lire les marqueurs

```
<<<<<<< HEAD
Route::get('/voyages/{voyage}/edit', ...);
Route::put('/voyages/{voyage}', ...);
Route::delete('/voyages/{voyage}', ...);
=======
Route::get('/voyages/create', ...);
Route::post('/voyages', ...);
>>>>>>> origin/main
```

`HEAD` : ma branche · après `=======` : ce que `main` a reçu.

Résoudre n’est pas choisir un camp

- Écrire la version qui respecte **les deux intentions**
- Ici : garder les cinq routes ; ailleurs, **un seul** bloc de message flash
- Supprimer les marqueurs, **tester**, puis `git add` et `git commit`
- `git push` : la PR redevient fusionnable

ACTIVITÉ

Premier tour : les trois parts du matin

À trois · chacun sa PR · revue en cercle

À produire : trois PR relues et fusionnées, `main` fonctionnelle.

1. Ouvrir sa PR, décrite selon le modèle
2. Relire : API → Création → Modification → API ; au moins un *Request changes* argumenté
3. Fusionner dans l'ordre API, Création, Modification ; le dernier résout le conflit

ACTIVITÉ

Second tour : de l'issue à la fusion

Chacun une issue · une branche · une PR avec `Closes #n`

À produire : trois issues fermées par leur PR, un conflit résolu par chacun.

1. #7 Destination sur la carte : Modification
2. #8 Durée sur la carte : API
3. #9 Nombre de classes sur la carte : Création

Chacun ajoute un `<li>` **en tête** de la liste de `voyage-card` : fusion dans cet ordre, l'API puis la Création intègrent `main` et résolvent.

À VOUS

Restitution

Chacun, sur son poste projeté :

1. Sa PR : commentaire reçu, commentaire donné
2. Son conflit : les deux versions, celle retenue, pourquoi
3. `git log --oneline --graph` autour de sa fusion

Puis : clone neuf, installation devant l'équipe. **Combien de commandes ? Qu'est-ce qui change d'un poste à l'autre ?**

À RETENIR

À retenir

- Une part = une **branche courte** = une **PR relue**
- `main` est protégée par une règle, pas par la bonne volonté
- Un conflit se résout **en comprenant les deux intentions**, puis se teste
- La PR sera bientôt le point d'entrée des **tests automatiques**

Intervention 3 : une application qui démarre pareil partout, avec Docker.